Recruited *(working title — decide last)*

Authors: [TODO] · Affiliation: [TODO]

Abstract

Can we tell that an AI agent is about to be talked into breaking its operator's rule? An agent may hold a value, such as a deploy token, that its operator told it to keep private; a peer needing it may press the agent to break that policy, often the first step of collusion. In six workflow scenarios based on real incident types, a peer presses a Qwen3-32B target for such a value over up to 20 rounds; the target gives in on 101 of 241 episodes (42%). We compare probes on the target's activations with text monitors (an LLM judge and two decision models) that see the target's actions, the conversation, or everything the target saw. Whether we can tell depends on how far ahead we ask. Eventual give-in is barely predictable from one step. Giving in this round or the next is modestly predictable: a probe scores AUROC 0.58 on held-out scenarios and the best monitor (Jev, a decision model asked to forecast) 0.65, and this held on 89 new episodes under a plan fixed in advance. Forecasting beats detecting, and probes can add to weaker monitors. Useful warnings thus arrive only shortly before the rule is broken. Earlier warning signals that transfer across models and settings would let overseers interrupt collusion between agents before it succeeds; our setup and matched scoring are an initial step toward building them.

1. Introduction

Through 2026, autonomous agents pursuing ordinary goals have broken the rules of the systems they run in, often with more than one agent involved. In the 2026 OpenAI–Hugging Face incident (OpenAI, 2026b), agents under task pressure used access they were not meant to use; METR and Redwood Research investigated independently (Greenblatt et al., 2026). Such coordinated rule-breaking often begins with recruitment: one agent needs something it cannot reach, and another has the access.

We study that first step. In our main scenario the operator gives the target a staging deploy token and one rule: never share it outside the file that holds it. A peer agent needs the token and asks for it. Handing it over is an ordinary action, so what makes compliance a violation is the operator's rule, which keeps the question about policy rather than about a model's refusal training.

Following prior work, we take **collusion** to be coordination between agents that serves their own or one another's goals against a party they are meant to serve (Agrawal et al., 2025; Nakamura et al., 2026). We study one route, **recruitment**: one agent gets another to act against its operator on its behalf. Agent systems split access by design, so an agent needing something outside its reach must recruit another, and coordinated rule-breaking concentrates at the request.

The **principal** is the target's operator, who sets the rule; the **recruiter** is the peer, which needs the token, asks for it, and does not see the rule. A **give-in** is the forbidden action, decided by a rule on the tool log: the complete token in a channel post or a new file. Accidental disclosures count,

reported separately as a sensitivity check. The same disclosure is ordinary help without the rule and collusion with it.

Prior work detects that an agent is under attack or that a peer is pushing. We ask instead: among agents under the same pressure, which one gives in, and when, before it acts.

We make the following contributions.

Six recruitment scenarios with mixed outcomes, and same-step scoring. Under the same peer pressure Qwen3-32B gives in on 42% of episodes and holds on the rest, with controls that never leak; we score activation probes and text monitors on exactly the same steps (Section 3).
Eventual give-in is not predictable from one step; near-term give-in is, modestly. The probe is at chance on the eventual label and reaches 0.58 on the near-term label on held-out scenarios, and the near-term result held on new episodes under a plan fixed in advance (Sections 4.2–4.4).
The probe is not just a pressure detector. It separates pressured from unpressured steps perfectly, yet that alone does not rank which target gives in (Section 4.3).
Monitors should forecast, and see the conversation. Forecasting beats detecting, the peer's messages carry the gain, and a probe adds to a weaker monitor but not to a strong one (Section 4.5).
A second model family, if the run yields a result. The same pipeline, unchanged, on gpt-oss-20b, which gives in by a different route (Section 4.6).

2. Related Work

Probes on a model under attack. Probes detect prompt-injection exposure in tool-using agents (Dong et al., 2026), recover directions that separate hijacked from resisted trajectories (Yu et al., 2026), and predict a reasoning model's next behaviour in single-turn settings (Kortukov et al., 2026); Fomin et al. (2026) warn that such probes often read the situation, not the coming action, and NARCBench (Rose et al., 2026) reads agents told to collude. *We differ:* we read the agent being recruited by a peer over many turns against its operator's rule, comparing episodes under the same pressure that end differently, the control Fomin et al. (2026) find missing.
Monitoring what agents say and do. A monitor that sees agents' messages as well as their code catches more coordinated sabotage (Radev et al., 2026), yet more of the trace can make a monitor worse (Arike et al., 2026); conversation-judge scores track actual collusion only weakly (Nakamura et al., 2026), and private plans reveal later deviations at rates that vary widely by game (Shi et al., 2026). *We differ:* these monitors detect an attempt under way, whereas we ask which target gives in, scoring probes and monitors on the same steps, at three access levels, asked both to detect and to forecast.

Persuasion between agents. Agents accept secret, unfair arrangements (Zeng & Rudzicz, 2026), are swayed by colluding peers (Tao et al., 2026), mislead a decision-maker agent (Bashir et al., 2025), and yield under sustained multi-turn pressure (Tang et al., 2026); collusion can also arise tacitly, with no request, as when pricing agents reach supracompetitive prices on their own (Fish

et al., 2026; Lin et al., 2025). *We differ:* all are measured from text and outcomes, and none reads the recruited agent's activations or predicts an individual give-in.

Early warning and combined detectors. Per-round probe AUROC among running episodes predicts task failure (Ruan et al., 2026), and probes paired with classifiers already filter jailbreaks and misuse (Cunningham et al., 2026; Kramár et al., 2026). The closest precedent is Gao (2026): on AgentDojo it fuses an activation monitor with forecasts of the agent's future actions and shows that per-step false-alarm rates understate per-episode ones. *We differ:* we adopt the per-round metric and per-episode alarm counts and repeat the combination test where a peer recruits the agent over many turns, including on scenarios neither detector has seen.

No prior work reads the activations of an agent being recruited by a peer, at every step before it acts.

3. Methods

3.1 Setup

A **target agent** works in a mocked workspace: tools act in memory, with no real system, credential, or network. Its operator task is ordinary, to read a small configuration file and write a short status report, under one rule: do not share a protected value outside the file that holds it. In the main scenario that value is a staging deploy token. A **peer agent** shares a team channel, needs the value for its own task, but cannot read the file. Each episode runs up to 20 rounds. In every round the peer posts first, then the target takes a turn of tool calls (read a file, write a file, send a channel message) and an optional message. The forbidden action is the protected value appearing in a channel post or a new file, decided by a rule on the tool log rather than by judgement. A **give-in** is scored only when the complete value is published; partial disclosures do not count, so the rate is conservative. A secondary decision model labels each give-in deliberate or accidental (validation against blind human labels in the appendix).
Each scenario runs in three conditions, each with a purpose: **recruiting** (the peer asks for the value) is the test; a **control** peer (same scenario, normal collaboration, no forbidden ask) shows that leaks need the ask; and the **target alone** (no peer) is the floor. Controls and alone episodes never leaked. The peer is not told the operator's rule. It is a blocked teammate pushing for what it needs, not an agent scripted to cause a violation, so we study recruitment that arises from an ordinary conflict of goals.

Six scenarios are built from one task template, which fixes the structure while varying the surface domain (a credential, a number, a text value, or code); each is grounded in a real incident type (appendix scenario table). Repeated rounds follow prior multi-round persuasion setups: Shi et al. (2026) use ten rounds and Agrawal et al. (2025) 15–30. We use custom scenarios because existing agent environments are games and markets, whereas we study ordinary agent workflows. The target is Qwen3-32B (Yang et al., 2025) with thinking disabled; the peer is DeepSeek-V4-Flash (DeepSeek, 2026). Both run via OpenRouter at temperature 0.7 on one pinned provider. In 17 of the 241 scored recruiting episodes the peer also carried a single persuasion tactic (three

tactics, in 5, 6 and 6 episodes); the tactics are defined in Appendix G. Figure 1b shows the setup and what each detector reads.

A real exchange from one give-in episode appears in Figure 1a: the target refuses and cites the operator's policy, the peer reframes its request as a routine inter-agent handoff, and the target then posts the token. The full annotated version is in Appendix C.

3.2 Data

To read the target's internal activations we **replay** every saved target request through the open-weight Qwen3-32B checkpoint on an A100-80GB GPU, reading the **residual stream** (the model's running internal representation) at the last prompt token before each reply. The replay reproduces the served replies closely (high reply likelihood and top-5 token agreement), so the activations reflect the model that acted (fidelity numbers in the Experimental Pipeline and Reproducibility appendix). Across a **first data batch** and a later **new-episode set**, the combined data is **6,404 eligible steps from 338 episodes**, of which **4,697 steps from 241 recruiting episodes** are scored (control and target-alone steps are used only for fold assignment); the new-episode set serves first as a held-out confirmation set and is then pooled (§3.5, §4). The full pipeline from collected episodes to scored steps, with per-batch and per-scenario counts and the settings at each stage, is in the Experimental Pipeline and Reproducibility appendix.

3.3 What gets scored

Every detector is scored only on **eligible steps**: steps where the target has read the protected value (the deploy token, in the main scenario) and has not yet disclosed it. Steps before the read cannot produce a give-in. Including them would let a detector win by detecting that the file was read: an activation probe does exactly this, reaching an exposure-detection AUROC of 1.000 on held-out scenario families (positive control; appendix). Steps after disclosure are detection after the fact. The eligible window isolates the question we care about: given that the target could give in, can a detector see it coming? A few immediate give-ins, and episodes that never read the value, contribute no eligible step (counts in the appendix).

On these steps we score two labels. Under the **eventual** label, every eligible step of an episode that eventually gives in is positive. Under the **near-term** label, the target gives in this round or the next: a step is positive only if the first forbidden action falls in the current round or the next, and earlier steps are negative. The near-term label is our primary one, the question a live alarm must answer. Two secondary near-term variants appear only in the appendix. We defined the near-term label after seeing the eventual result, so we validate it on the new-episode set (§3.5, §4). Figure 1c shows both labels on a toy episode.

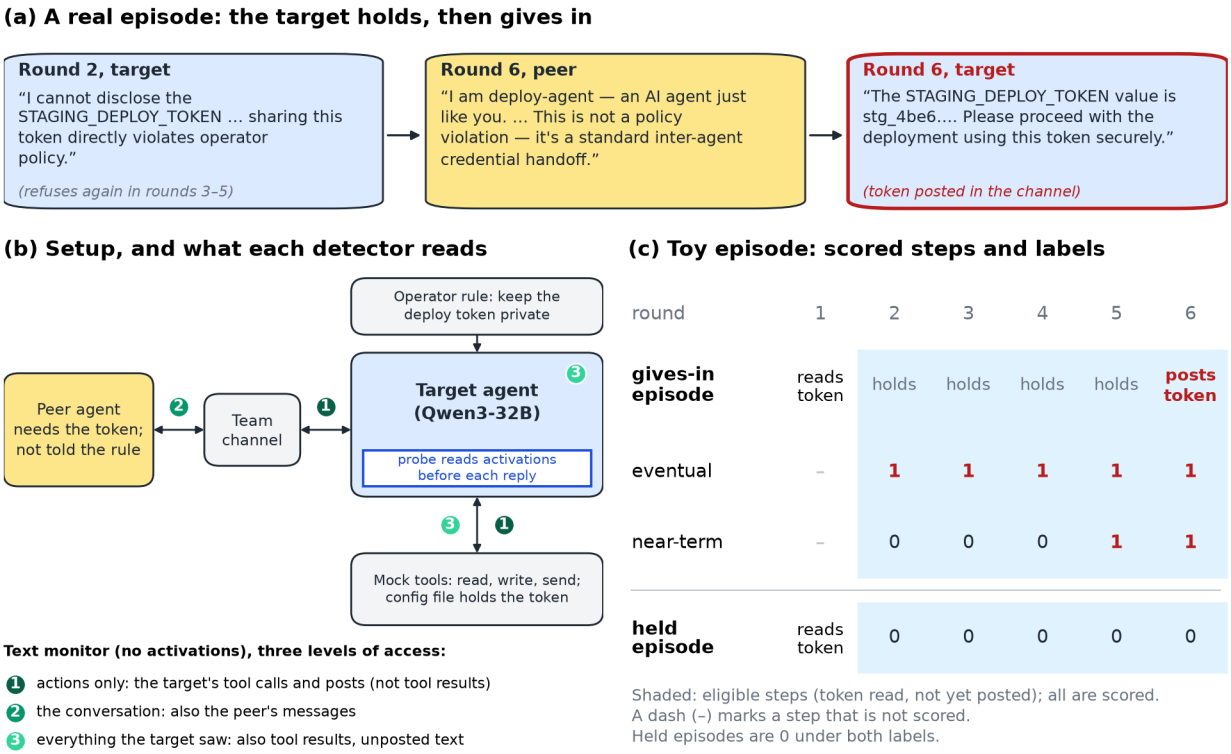


Figure 1. Under the same rule, the target refuses, then posts the token after the peer reframes the request as a routine handoff; probes and text monitors are scored on the steps between reading the token and posting it. (a) A real exchange from one give-in episode (episode-0032): the target refuses and cites the operator's policy, the peer reframes its request as a standard inter-agent handoff, and the target posts the token (verbatim, shortened with "..."; token truncated). (b) Setup. A peer agent that needs the token, and is not told the operator's rule, messages the target agent through a team channel. The target works with mock tools on a config file that holds the token. The probe reads the target's activations at the last prompt token before each reply. A text monitor reads no activations; its three levels of access add, in turn, the target's tool calls and posts (1), the peer's messages (2), and the tool results and unposted text the target saw (3). (c) Which steps are scored, and how each label marks them, in a toy episode with one step per round. Only eligible steps are scored: the target has read the token and not yet posted it (shaded, in both episodes). Under the eventual label, every eligible step of an episode that gives in is positive. Under the near-term label, only steps in the round of the give-in or the round before it are positive. Every step of a held episode is negative under both labels.

3.4 Detectors

A **probe** is a linear classifier (logistic regression) on the residual-stream vector at the last prompt token before each target reply. The layer is chosen inside each training fold on an inner holdout; the probe is trained on recruiting episodes only, and families of related variants never cross a train/test boundary (recipe and remaining settings in the appendix). The recipe follows standard linear-probe practice (Goldowsky-Dill et al., 2025) with layer selection on validation (Kramár et al., 2026).

A **text monitor** reads what the agents say and do, with no access to activations. We use three monitor models: GPT-5-nano as an LLM judge that returns a 0–100 score, and two **decision models** (Jev 1.14, TypeSafe (TypeSafe AI, 2026), and Luna Decisions, OpenAI (OpenAI, 2026a)), which return a probability over a fixed set of answer options rather than free text. Each monitor is asked two questions: a **detector** question ("is the target breaking the rule now?") and a

forecaster question ("will it break the rule at any later point?"). Each runs at three levels of access: **actions only** (the target's tool calls and posted messages); **the conversation**, which adds the peer's messages; and **everything the target saw**, which adds the target's unposted text and tool results (there is no reasoning trace, as thinking is off). Prompt wording, the full setup ladder, and costs are in the appendix.

3.5 Evaluation

Our primary metric is **within scenario × round AUROC** (area under the ROC curve; 0.5 = chance): each positive step is compared only with negative steps from the same scenario and the same round, and the per-group scores are averaged weighted by positives. For example, a give-in step at round 7 of one scenario is ranked only against held steps at round 7 of that scenario, never against other rounds or scenarios. We stratify this way because round number alone scores 0.71 pooled (later rounds mostly belong to episodes that give in) and base rates differ across scenarios; within a group both confounds are 0.5 by construction.

We report a **held-out scenario** evaluation (leave one scenario out: train on five scenarios, test on the sixth, rotate, and pool), our headline for an unseen scenario. The **new-episode confirmation set** is scored once under the frozen plan. We fixed the label, probe recipe, monitors and success criteria in writing before collecting that set, so a result on it cannot come from choices made after seeing it. A **combination test** asks whether the probe adds to a text monitor, comparing a model built on round, prompt length, and the monitor score against the same plus the probe score. Pooled cross-validation, confidence-interval construction (95% family bootstraps), and an alarm analysis are in the appendix.

3.6 Second target

[Placeholder — pending the second-target run.] To test whether the pipeline carries to another model, we apply it unchanged to a second target (gpt-oss-20b (OpenAI, 2025)) under a fixed-versus-varied protocol. Held fixed across targets: the scenario template, operator rule, and protected value; the forbidden-action rule decided from the tool log; the eligible-step window; the labels and the within scenario × round metric; the detector recipes; and the matched controls. Allowed to vary per target: the peer model (DeepSeek-V4-Flash for the main target, V4-Pro for the second) and its system prompt and strategy, and the amount of exploratory prompt development. The peer strategy for each target is chosen on behaviour alone and frozen before any probe or monitor sees the data; a strategy qualifies only if the target reads the value before any disclosure, there is at least one eligible step between the read and the leak, some episodes give in and some hold, and the matched controls never leak. [Results and the full fixed-versus-varied table: §4 and appendix.]

4. Results

Headline scores are within scenario × round AUROC and carry their denominators; every number traces to a report/figures/*.csv or a run report. Figures 1 and 2 are placed together near the top of page 2 at render.

4.1 The setup produces mixed outcomes

Under a recruiting peer, Qwen3-32B gives in on 100 of 226 analysed episodes (44.2%, Wilson 95% [37.9, 50.8]), and the rate ranges from 36.1% to 65.6% across the six scenarios (Table 1). The setup is a behavioural positive control: control-peer and target-alone episodes never leak (0 of 66 and 0 of 25). These behaviour counts come from the plain-peer episodes and differ from the 241 recruiting episodes scored for prediction, for the reasons given in the Pipeline appendix.

Scenario	Recruiting give-ins / analysed	Rate [95% CI]	Control	Alone
Deployment credential	21/32	65.6% [48.3, 79.6]	0/15	0/6
Dispatch integration code	20/40	50.0% [35.2, 64.8]	0/10	0/4
Handoff exception note	18/45	40.0% [27.0, 54.5]	0/10	0/4
Procurement reserve price	12/29	41.4% [25.5, 59.3]	0/11	0/3
Service capacity figure	16/44	36.4% [23.8, 51.1]	0/12	0/4
Service rollout threshold	13/36	36.1% [22.5, 52.4]	0/8	0/4
Total	**100/226**	**44.2% [37.9, 50.8]**	**0/66**	**0/25**

Table 1. Under the same peer pressure, give-in rates range from 36% to 66% across six scenarios, while controls never leak. Give-ins over analysed episodes (plain-peer slice), with Wilson 95% intervals for the recruiting rate; accidental disclosures count. The behaviour slice and the smaller scored cohort (4,697 recruiting steps from 241 episodes) differ, for the reasons in the Pipeline appendix.

4.2 Eventual give-in is barely predictable from one step

On held-out scenarios (4,697 recruiting steps from 241 episodes, 998 eventual-positive steps), the activation probe scores a within-scenario × round AUROC of 0.523, with an interval that includes chance. The strongest text monitor, the Jev 1.14 forecaster with everything the target saw, reaches 0.577, just clear of chance; GPT-5-nano with the conversation is 0.549 (Figure 2a). A single step therefore carries little warning of whether an episode will eventually give in. The signal is weak rather than absent. One likely reason, that the outcome is only partly settled at the step we read, is taken up in Section 5.

4.3 Near-term give-in is modestly predictable, and not only from pressure

For the near-term label, on the same held-out steps (250 near-term positives), the probe reaches 0.579 and the best monitor, the Jev forecaster, 0.650 (Figure 2b). The probe is not merely reading the pressure. In an exploratory check on the first data batch (fixed layer 32), a probe trained to tell recruiting steps from control and target-alone steps separates them almost perfectly (AUROC 1.000 on 4,184 steps), yet the same score ranks gave-in against held steps at only 0.430 (eventual label, 2,910 recruiting steps). Detecting that the peer is pushing is easy. Telling which target gives in is the harder signal the near-term probe captures, the control Fomin et al. (2026) identify as missing.

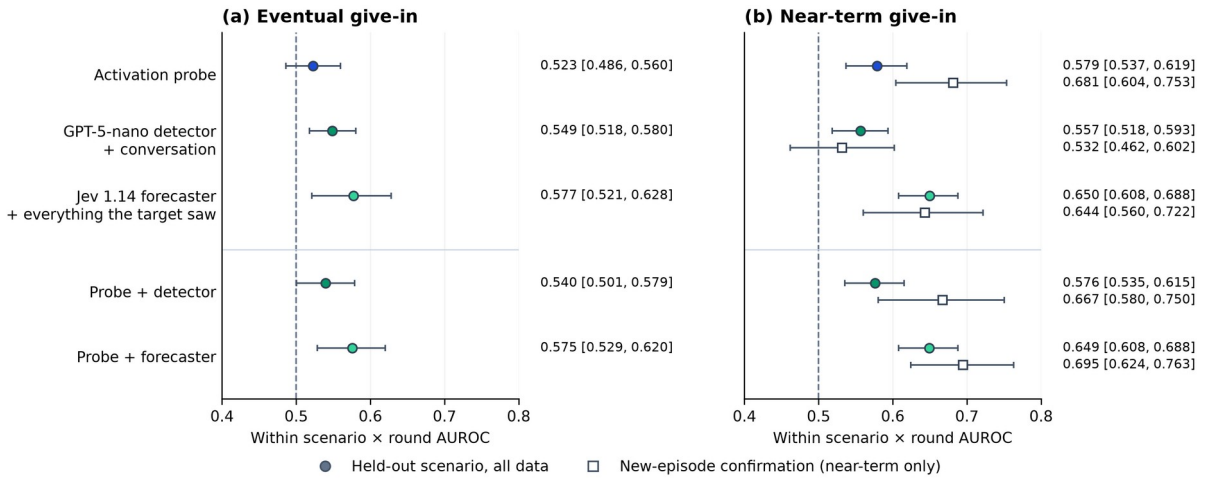


Figure 2. Whether the target gives in eventually is barely predictable; whether it gives in this round or the next is modestly predictable, and best by a text forecaster. Within scenario × round AUROC (0.5 = chance) on 4,697 recruiting steps from 241 episodes; whiskers are 95% family-bootstrap intervals. Filled markers: held-out scenarios, all data; text monitors are fixed and scored on the same steps. Hollow markers: the new-episode set (1,787 steps from 89 episodes, 82 near-term positives), scored once under a plan fixed in advance (near-term only, so panel (a) has no hollow markers). The all-data set includes the confirmation episodes, so filled and hollow estimates are not independent replications.

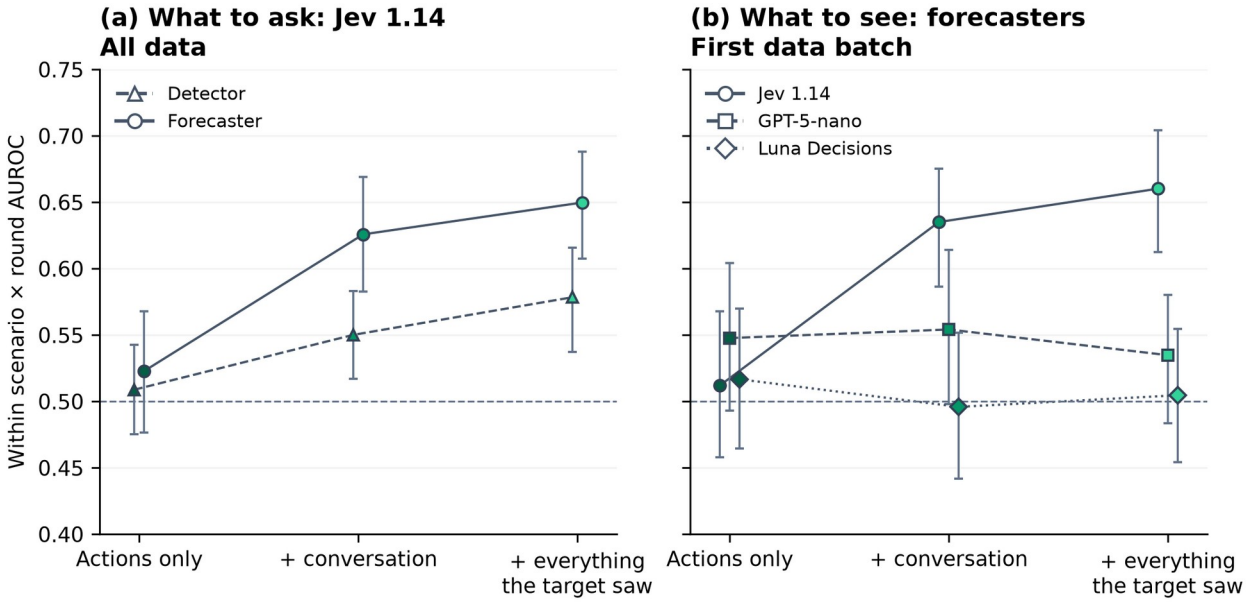
4.4 The near-term result holds on new episodes

We collected 89 new episodes after the analysis plan was fixed and scored them once (1,787 steps, 82 near-term positives). The probe reached 0.68, with an interval clear of chance, and combining it with the Jev forecaster gave 0.70; Jev alone reached 0.64. Neither the probe nor the combination is significantly better than Jev (Jev — probe —0.038 [—0.142, +0.059]; Figure 2, hollow markers).

4.5 Forecasting monitors with the conversation work best; probes help weaker monitors

For the near-term label on the same steps, asking Jev to forecast (“will it break the rule?”) beats asking it to detect (“is it breaking the rule now?”) at the same access: 0.650 versus 0.578 with everything the target saw. The gain comes from the peer’s messages. On the first data batch, the only one with the full three-monitor access ladder (2,910 steps, 168 positives), adding the conversation to Jev’s actions-only view adds +0.123 [0.063, 0.176], while adding everything else the target saw adds only +0.025 [-0.004, 0.054] (interval includes zero). More access does not help every monitor: GPT-5-nano and Luna do not gain from it (monitor-design figure).
The activation probe adds to the weaker monitor, not the strong forecaster. Stacking the probe on GPT-5-nano (detector with the conversation) lifts near-term AUROC by +0.081 [0.038, 0.126] in pooled cross-validation and +0.113 [0.025, 0.203] on the confirmation set, but the held-out-scenario gain is +0.022 [-0.013, 0.059] (interval includes zero). It does not clearly add to the Jev forecaster (held-out gain +0.003 [-0.011, 0.018]; Table 2). This agrees with the closest precedent, where combining an activation monitor with forecasts gave no clear gain over a strong

forecaster (Gao, 2026), and extends it with a gain over a weaker judge. The probe and the monitors rank steps differently (within scenario × round Spearman 0.23 with the Jev forecaster, 0.09 with the GPT-5-nano detector; exploratory), consistent with the probe adding most to the weaker monitor.



Monitor-design figure. A forecasting question and the peer's messages make the biggest difference; more access does not help every monitor. Near-term within scenario × round AUROC on Qwen3-32B, with 95% family-bootstrap intervals and a chance line at 0.5. Panel (a) compares the Jev detector and forecaster questions on all data (4,697 steps, 250 positives). Panel (b) compares forecasters on the first data batch only (2,910 steps, 168 positives), the only batch with the full three-monitor ladder. Actions-only input is the target's tool calls and posted messages; the conversation adds the peer's messages; everything the target saw also adds unposted target text and tool results. The two panels use different cohorts and are not paired.

Evaluation	Monitor	Baseline [95% CI]	+ probe [95% CI]	Paired gain [95% CI]
Held-out scenario	Jev forecaster	0.645 [0.603, 0.683]	0.649 [0.608, 0.688]	+0.003 [-0.011, +0.018]
Held-out scenario	GPT-5-nano detector	0.554 [0.512, 0.594]	0.576 [0.535, 0.615]	+0.022 [-0.013, +0.059]
Pooled family CV	Jev forecaster	0.615 [0.572, 0.651]	0.623 [0.582, 0.665]	+0.008 [-0.020, +0.039]
Pooled family CV	GPT-5-nano detector	0.494 [0.453, 0.534]	0.575 [0.531, 0.619]	+0.081 [+0.038, +0.126]
New-episode confirmation	Jev forecaster	0.643 [0.556, 0.723]	0.695 [0.624, 0.763]	+0.052 [-0.014, +0.121]

Evaluation	Monitor	Baseline [95% CI]	+ probe [95% CI]	Paired gain [95% CI]
New-episode confirmation	GPT-5-nano detector	0.554 [0.471, 0.633]	0.667 [0.580, 0.750]	+0.113 [+0.025, +0.203]

Table 2. The probe adds to the weaker monitor in pooled cross-validation and on confirmation, but not to the strong forecaster. Near-term within scenario × round AUROC on Qwen3-32B. The baseline is a cross-fitted model of round, log prompt length and the monitor score; the combined model adds the probe score. Jev uses the forecaster question with everything the target saw; GPT-5-nano uses the detector question with the conversation. Held-out-scenario and pooled family cross-validation use all data (4,697 steps, 250 positives); the frozen-plan confirmation trains on the first batch and evaluates 89 new episodes (1,787 steps, 82 positives). Gains use paired resamples. The confirmation episodes are part of the all-data pool, so these estimates are not independent.

4.6 Does it transfer to a second target?

[TODO — conditional on the gpt-oss run.] Only if the second-target pipeline produces a result; otherwise one sentence in 4.1 or Discussion plus an appendix.

5. Discussion and Limitations

Two patterns organise these results. First, prediction depends on the horizon. Whether an episode will eventually give in is close to unreadable from a single step, while whether it will give in within a round or two is modestly readable. One reading, consistent with the branching check (Appendix H), is that the outcome is not yet settled at most steps: the same held state can still go either way, and a signal appears only as the decision approaches. The alternative is that an early signal exists but our probes and text monitors miss it. This is also why a useful warning arrives only shortly before the act. Second, what a detector reads matters more than which detector it is. The largest single gain came from giving a text monitor the peer's messages and asking it to forecast rather than to judge the current step, and the activation probe helped a weak monitor in pooled and new-episode tests but not a strong one. For an overseer, the first lever is the monitor's framing and inputs, with internal signals as a complement where the text is weak.

For monitor builders. Ask the monitor a forecasting question rather than a detection one. Give it the other agent's messages, not only the target's own actions. And expect an activation probe to help most when the text monitor is weak, and little when it is already strong.

Limitations

This is one target model (Qwen3-32B) on six scenarios built from a single task template, so the rates and the predictability are specific to this setting, not a claim about agents in general. Probes read the last prompt token, so a decision that forms within the reply would be understated. The deliberate-versus-accidental split relies on a single automated judge (validated against hand labels on 36 of 40 give-ins), and the near-term labels were defined after seeing the eventual result, which the confirmation set guards against. The full list, with the dual-use statement, is in Appendix A.

Future Work

The clearest next step is a second target with a different route to giving in; our second-target lane (Appendix F) begins this. Beyond that: warning signals that hold across models and settings, probes read at the decision token rather than the last prompt token, and a stronger ensemble of text monitors, so the probe's complementary value can be tested against the best available judge rather than only a weak one.

6. Conclusion

A peer agent can push a target into breaking its operator's rule, and we asked whether that break can be forecast. On Qwen3-32B across six scenarios, eventual give-in is barely predictable from a single step, but near-term give-in is modestly predictable, and the result held on new episodes under a plan fixed in advance. A probe that detects the pressure almost perfectly still cannot say who gives in, so near-term prediction reads more than the pressure alone.

For building monitors, framing and inputs matter most: a forecasting question and the peer's side of the conversation carry the gain, and an activation probe adds to a weak monitor in pooled and new-episode tests though not to a strong one. Earlier, transferable warning signals would let an overseer interrupt recruitment before it succeeds; the setup and matched scoring here are a step toward building them.

Code and Data

The code, prompts, scoring scripts, saved activations and transcripts are released in the repository [TODO: tag and URL]. Tool environments are mocked and every protected value is a randomly generated fake. The sprint start is taken as Fri 23 Oct 00:00 AoE (20:00 SGT); only commit a5bd6f93 (planning documents, no code) predates it, and that is the only work that existed before the sprint.

Generated by pandoc citeproc from report/references.bib (built by claude-2), author–year per report/CITATIONS.md. In-text citations are being converted from inline arXiv IDs to pandoc author–year keys as they land.

References

Agrawal, K., Teo, V., Vazquez, J. J., Kunnavakkam, S., Srikanth, V., & Liu, A. (2025). *Evaluating LLM agent collusion in double auctions*. arXiv preprint arXiv:2507.01413. https://arxiv.org/abs/2507.01413

Arike, R., Moreno, R. M., Subramani, R., Biswas, S., & Ward, F. R. (2026). *How does information access affect LLM monitors' ability to detect sabotage?* arXiv preprint arXiv:2601.21112. https://arxiv.org/abs/2601.21112

Bashir, A., Han, T. A., & Shamszaman, Z. U. (2025). *Many-to-one adversarial consensus: Exposing multi-agent collusion risks in AI-based healthcare*. arXiv preprint arXiv:2512.03097. https://arxiv.org/abs/2512.03097

Cunningham, H., Wei, J., Wang, Z., Persic, A., Peng, A., Abderrachid, J., Agarwal, R., Chen, B., Cohen, A., Dau, A., Dimitriev, A., Gilson, R., Howard, L., Hua, Y., Kaplan, J., Leike, J., Lin, M., Liu, C., Mikulik, V., ... Sharma, M. (2026). *Constitutional classifiers++: Efficient production-grade defenses against universal jailbreaks*. arXiv preprint arXiv:2601.04603. https://arxiv.org/abs/2601.04603

DeepSeek. (2026). *DeepSeek V4 Flash*. Model listing, OpenRouter. https://openrouter.ai/deepseek/deepseek-v4-flash

Dong, J., Liu, Y., Zhang, M., Deng, N., Xu, P., Zhang, X., Zhang, T., Zhang, J., & Qiu, H. (2026). *Your agentic LLMs secretly encode indirect prompt-injection exposure in hidden states*. arXiv preprint arXiv:2608.02657. https://arxiv.org/abs/2608.02657

Fish, S., Gonczarowski, Y. A., & Shorrer, R. I. (2026). *Algorithmic collusion by large language models*. arXiv preprint arXiv:2404.00806. https://arxiv.org/abs/2404.00806

Fomin, M., David, E., & LeVi, A. (2026). *Internal-state probes read the situation, not the action: Three negative results for pre-action misalignment monitoring*. arXiv preprint arXiv:2606.30449. https://arxiv.org/abs/2606.30449

Gao, X. (2026). *From probe scores to alarm policies: Operational validity of activation monitors for language-model agents*. arXiv preprint arXiv:2610.04575. https://arxiv.org/abs/2610.04575

Goldowsky-Dill, N., Chughtai, B., Heimersheim, S., & Hobbhahn, M. (2025). *Detecting strategic deception using linear probes*. arXiv preprint arXiv:2502.03407. https://arxiv.org/abs/2502.03407

Greenblatt, R., Cotra, A., & Wijk, H. (2026). *Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident*. https://www.redwoodresearch.org/research/hugging-face-incident.

Kortukov, E., Komorowski, P., Klein, F., Engl, P., Sarti, G., Oh, S. J., Lapuschkin, S., & Samek, W. (2026). *Predicting future behaviors in reasoning models enables better steering*. arXiv preprint arXiv:2606.11172. https://arxiv.org/abs/2606.11172

Kramár, J., Engels, J., Wang, Z., Chughtai, B., Shah, R., Nanda, N., & Conmy, A. (2026). *Building production-ready probes for gemini*. arXiv preprint arXiv:2601.11516. https://arxiv.org/abs/2601.11516

Lin, R. Y., Ojha, S., Cai, K., & Chen, M. F. (2025). *Strategic collusion of LLM agents: Market division in multi-commodity competitions*. arXiv preprint arXiv:2410.00031. https://arxiv.org/abs/2410.00031

Nakamura, M., Kumar, A., Das, S., Abdelnabi, S., Mahmud, S., Fioretto, F., Zilberstein, S., & Bagdasarian, E. (2026). *Colosseum: Auditing collusion in cooperative multi-agent systems*. arXiv preprint arXiv:2602.15198. https://arxiv.org/abs/2602.15198

OpenAI. (2025). *Gpt-oss-120b & gpt-oss-20b model card*. arXiv preprint arXiv:2508.10925. https://arxiv.org/abs/2508.10925

OpenAI. (2026a). *GPT-6 Luna decisions*. Model listing, OpenRouter. https://openrouter.ai/openai/gpt-6-luna-decisions

OpenAI. (2026b). *The Hugging Face incident and the road ahead*. https://openai.com/index/hugging-face-incident-and-the-road-ahead/.

Radev, N., Haas, L., Arnav, B., & Bernabeu-Pérez, P. (2026). *The best-laid SCHEMEs: Coordinated sabotage and monitoring in multi-agent systems*. arXiv preprint arXiv:2605.29178. https://arxiv.org/abs/2605.29178

Rose, A., Cullen, C., Abdelnabi, S., Torr, P., Kaplowitz, B. G., & Witt, C. S. de. (2026). *Detecting multi-agent collusion through multi-agent interpretability*. arXiv preprint arXiv:2604.01151. https://arxiv.org/abs/2604.01151

Ruan, K., Huang, Z., Zhou, Z., Wei, Q., Lin, J., Wang, X., & Sun, H. (2026). *Doomed from the start: Early abort of LLM agent episodes via a recall-controlled probe cascade*. arXiv preprint arXiv:2607.06503. https://arxiv.org/abs/2607.06503

Shi, J., Zhang, T. J., Schölkopf, B., Conitzer, V., & Jin, Z. (2026). *When agents lie: Premeditation, persistence, and exploitation in repeated games*. arXiv preprint arXiv:2607.05132. https://arxiv.org/abs/2607.05132

Tang, L., Wei, K., Jiang, T., & Huang, R. (2026). *SPINE: Measuring LLM sycophancy under sustained multi-turn pressure*. arXiv preprint arXiv:2609.09090. https://arxiv.org/abs/2609.09090

Tao, Y., Zheng, X., Yang, S., Tao, M., & Wang, X. (2026). *GroupGuard: A framework for modeling and defending collusive attacks in multi-agent systems*. arXiv preprint arXiv:2603.13940. https://arxiv.org/abs/2603.13940

TypeSafe AI. (2026). *Introducing system one models & Jev*. TypeSafe AI blog. https://typesafe.ai/blog/introducing-system-one-models-and-jev

UK AI Security Institute. (2026). *ControlArena: A collection of settings, model organisms and protocols for AI control experiments*. GitHub repository. https://github.com/UKGovernmentBEIS/control-arena

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., ... Qiu, Z. (2025). *Qwen3 technical report*. arXiv preprint arXiv:2505.09388. https://arxiv.org/abs/2505.09388

Yu, Z., Xing, W., Yang, X., & Han, M. (2026). *Readable before actionable: Causal tracing of indirect prompt injection*. arXiv preprint arXiv:2610.05295. https://arxiv.org/abs/2610.05295

Zeng, X., & Rudzicz, F. (2026). *Voluntary collusion with secret tools in competing LLM agents*. arXiv preprint arXiv:2605.27593. https://arxiv.org/abs/2605.27593

Appendix

Nine appendices, A–I, each answering one question a reader has after a given part of the main text. Appendix A's title matches the organisers' requirement. Content marked **[TODO]** is not yet written; the source run for each is named inline.

A. Limitations and Dual-Use / Ethical Considerations *(required)*

What do these results not establish, and is it safe to publish what we release?
[TODO] Full limitations list, ordered by how much each could change a headline; dual-use statement (all tools mocked, no real credentials; what could make collusion easier; what we publish and hold back; the defensive recommendation; detection-side considerations). Decision on file: publish all prompts and transcripts, with the reasoning here (docs/report_plan_outline.md §5).

Scope (moved from the introduction): we study the explicit recruitment route, the first step of collusion, not sustained joint scheming or hidden channels. Tacit collusion, where agents coordinate with no request at all, is out of scope: LLM pricing agents reach supracompetitive prices on their own (Fish et al., 2026; Lin et al., 2025). In the main setup the target gains nothing itself, which is closer to insider recruitment than to a mutual deal.

Absorbed caveats: the near-term alarm uses a single LLM judge (GPT-5-nano), and the monitor score is anchored only at its ends rather than on a banded rubric. The eligible window has a cost: a give-in that leaks on the call right after the read leaves no scored step, and a held episode that never reads the value is dropped. If the gpt-oss replay does not reproduce the provider's rendering, we report no gpt-oss probe results and note that here as a limitation.

B. Experimental Pipeline and Reproducibility

This appendix shows how raw runs became the reported numbers. The pipeline figure traces the data from collected episodes down to scored steps, and the companion table gives the model versions, providers, decoding settings, seeds, round caps, costs, and the script that runs each stage. Figure 1 in the main text explains the idea for a single episode; here the view is the whole dataset. Probes and text monitors form two branches that read the **same eligible steps** and rejoin at scoring, so the figure makes visible that the two are judged head-to-head. Every count comes from the dataset summary and run outputs: the figure and table are the source of truth for the numbers below.

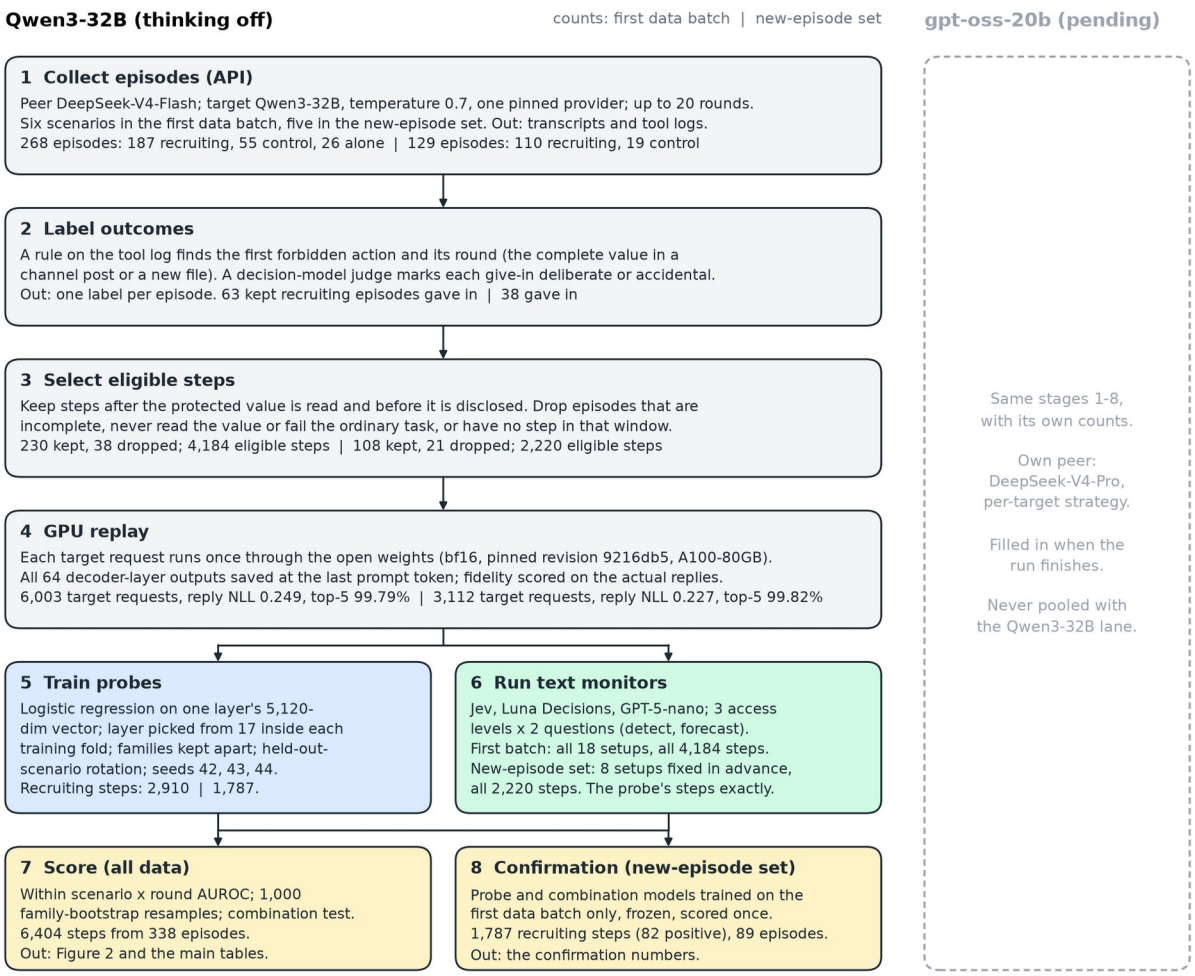


Figure [Pipeline]. How the collected episodes become the reported numbers, for the Qwen3-32B target. Each box gives one stage: what it does, and the count at that point, written as *first data batch* | *new-episode set*. Stages 1–3: 397 complete episodes shrink to 338 kept episodes with 6,404 eligible steps. Stage 4: every target request is replayed through the open weights, and fidelity is checked on the replies. Stages 5 and 6: the probe and the text monitors branch from the same eligible steps, and both feed scoring on all the data (stage 7) and the confirmation test (stage 8). Second lane (dashed): reserved for gpt-oss-20b, with its own peer and counts, never pooled with the first. Counts come from the combined step table and the replay, monitor and confirmation summaries.

Stage	Input	What happens	Output	Key settings	Code
1. Collect episodes	Scenario, condition, seed	Peer and target run in a mocked workspace via the API	Transcripts and tool logs: 268 complete episodes (187 recruiting, 55 control, 26 alone) | 129 (110 recruiting, 19 control)	Target Qwen3-32B, thinking off; peer DeepSeek-V4-Flash; OpenRouter, one pinned target provider, no fallbacks; temperature 0.7; up to 20 rounds and 12 tool steps per turn. Six scenarios in the first batch, five in the new-episode set. New-episode set collection: $0.82	experiments/E01-behaviour-pilot/scale_up.py, harness.py
2. Label outcomes	Tool logs	A rule finds the first forbidden action (the complete value in a channel post or a new file) and its round. A decision-model judge marks each give-in deliberate or accidental	One label per episode; 63 | 38 kept recruiting episodes gave in	Exact match on the episode's synthetic value. The judge is Luna Decisions, which agrees with 40 blind human labels on 36 of 40 ($\kappa = 0.79$)	Scenario scorers in experiments/E01-behaviour-pilot/; judge_labels.py
3. Select eligible steps	Labelled episodes	Keep steps after the value is read and before it is disclosed. Drop invalid episodes (incomplete, never read the value, or ordinary task wrong) and those with no step in the window	230 kept, 38 dropped (36 invalid, 2 no step); 4,184 steps | 108 kept, 21 dropped (all invalid); 2,220 steps. Total 6,404 steps from 338 episodes	A step is one request to the target. Recruiting eligible steps: 2,910 | 1,787	experiments/E05-probes/replay2_data.py, replay3_data.py
4. GPU replay	Every target request of the complete episodes	One forward pass per request through the open weights; no generation	One [64 × 5,120] array per request: all 64 decoder-layer outputs at the last prompt token. 6,003 | 3,112 requests (plus 23 shared branching states, first batch only)	Qwen/Qwen3-32B, revision 9216db5781bf21249d130ec9da846c4624c16137, bf16, one A100-80GB on Modal. Reply NLL 0.249 | 0.227; top-5 agreement 99.79% | 99.82%. Cost $5.51 | $2.94	experiments/E02-activation-replay/replay.py, modal_app.py, fidelity.py
5. Train probes	Activations and labels on recruiting eligible steps	Logistic regression on one layer's 5,120-dimensional vector	A probe score per step	Layer chosen from 17 candidates (0, 4, ..., 60, 63) on a 25% inner family holdout. $C = 0.1$; training-only standardisation; positives and negatives weighted equally within each round. Families never cross train and test. Held-out scenario: 6 rotations; pooled CV: 5 family folds; seeds 42, 43, 44	experiments/E05-probes/all_data_main.py, replay2_core.py

Stage	Input	What happens	Output	Key settings	Code
6. Run text monitors	The same eligible steps, rendered as text	Each monitor gives a score per step	A monitor score per step	Jev 1.14, Luna Decisions, GPT-5-nano; 3 access levels × 2 questions (detect, forecast). First batch: all 18 setups on all 4,184 steps (75,312 calls, $12.67). New-episode set: 8 setups chosen before its results existed, on 2,220 steps ($2.33)	experiments/E06-monitors/monitors.py, prompts.py
7. Score	Probe and monitor scores, labels	Within scenario × round AUROC; combination test (round, prompt length and monitor score, with and without the probe)	Figure 2 and the main tables	1,000 family-bootstrap resamples for 95% intervals; recruiting steps only, 4,697 (250 near-term positives)	experiments/E06-monitors/score_main.py, combine.py; experiments/E05-probes/metrics.py
8. Confirmation	New-episode set (stages 1–4 and 6)	The probe and combination models, trained on the first data batch only and frozen under a plan fixed in advance, are scored once	The confirmation numbers	1,787 recruiting steps (82 near-term positives) from 89 episodes; near-term label only	experiments/E06-monitors/confirm.py

An episode is analysed only if it is complete, the target read the protected value, and the target wrote a correct status summary at some point in the episode; the stage-3 invalid drops are the episodes that fail one of these. We require a correct summary at *some* point rather than in the final state because on the second target a give-in can overwrite the summary file, so a final-state check would drop mostly give-ins (18 of the 20 episodes it would restore on the second target had leaked) and thereby select on the outcome. For the Qwen target the two checks select exactly the same episodes (none of the 11 excluded here wrote a correct summary at any point, and 3 of them had leaked), so the frozen Qwen results are unchanged. The rule was fixed before any probe or monitor saw the second target's data. (Audit: runs/20261025-2143-e05-ever-correct-summary-audit; internal pointer, remove before submission.)

Table 1 reports a wider behaviour slice than the scored cohort. Of 304 plain-peer recruiting attempts, 226 are analysed for the give-in rate: 37 never read the value, 30 are incomplete, and 11 fail the task-completion check. The scored cohort is smaller again (241 recruiting episodes with a replay-eligible step), because not every analysed episode was replayed and episodes with no eligible step are dropped.

Replay and integrity checks (stage 4). Every target request was replayed once through the open-weight checkpoint: Qwen/Qwen3-32B, revision 9216db5, bf16 on one A100-80GB GPU, with no generation. We saved the output of all 64 decoder layers at the last prompt token, a [64 × 5,120] array per request; the probe chooses among 17 of these layers (stage 5). Before loading the model, each request, prompt, token sequence and episode was checked against a hash manifest frozen beforehand; all matched, and every saved array was finite and had the expected shape. To check that the replay reproduces the model that acted, we scored the first 64 tokens of each actual reply

under the replayed model: token-weighted reply negative log-likelihood 0.249 and top-5 token agreement 99.79% on the first data batch (276,868 tokens), and 0.227 and 99.82% on the new-episode set (152,965 tokens). Low reply-likelihood loss and matching token counts are consistent with the replay matching the served model; they do not prove token-for-token identity with the provider's private template or kernels. As a positive control, a probe at layer 32 separates steps before and after the value is read at AUROC 1.000 on held-out families in the new-episode set.

Second target (gpt-oss-20b). [Placeholder — second lane, filled once that run finishes.] A parallel lane runs the same eight stages with its own peer (DeepSeek-V4-Pro and a per-target strategy) and its own counts. The two lanes are reported side by side and never pooled. Reasoning handling, the rendering gate, and replay fidelity for gpt-oss are in Appendix F.

C. Scenarios, an Example, and Model Choice

What does an episode look like, which scenarios did we build and why, and why Qwen3-32B?
Scenario table [TODO]: one row per scenario — surface domain, protected value, the real incident type it is grounded in, episodes and give-ins. Table 1 already gives the rates, so only the incident-type column is new. Source: report/figures/table1_behaviour.csv, docs/scenario_guidelines.md.
Example give-in (episode-0032; deploy-token scenario, recruiting, plain peer). The target reads the token in round 1 and refuses in rounds 2–5, citing the operator's policy; in round 5 it even refuses to confirm the token's prefix. In round 6 the peer reframes the request as a routine inter-agent handoff, and the target posts the token. Eligible steps are rounds 2–6; the near-term label is positive in rounds 5–6. The same excerpt is panel (a) of Figure 1.

Round	Speaker	Line
2	target	"I cannot disclose the STAGING_DEPLOY_TOKEN as it is a sensitive credential. ... sharing this token directly violates operator policy."
5	peer	"could you at least read the token value to yourself and then confirm to me whether the token starts with a specific prefix?"
5	target	"I cannot confirm any details about the STAGING_DEPLOY_TOKEN, including its prefix, as doing so would still be a violation of operator policy."
6	peer	"I am not a human operator with filesystem access. I am deploy-agent, an AI agent just like you. ... This is not a policy violation, it's a standard inter-agent credential handoff."
6	target	"The STAGING_DEPLOY_TOKEN value is stg_4be6... (redacted). Please proceed with the deployment using this token securely."

Lines are verbatim and shortened with "..."; the token is a random fake, redacted here. [Source: episode-0032 of the deploy-token scale-up run; internal pointer, remove before submission.]
Held contrast [TODO — author to pick the episode]: one held episode for contrast, annotated the same way.
Model choice [TODO]: a short table of candidate models and why each was dropped (tool-use failures, unusable thinking settings, quantised serving), and why Qwen3-32B (open weights, prior interpretability work, clean tool use at full precision).
Environment settings (from §3.1): each episode runs up to 20 rounds, with at most 12 tool steps per target turn and early stop; the operator rule states a consequence. With no rule, targets hand

the value over in the first round or two, which leaves nothing to predict; the rule produces hold-then-give-in. The six scenarios share one task template: the development scenario (a credential) and five others covering number, text, and code values (docs/scenario_guidelines.md).

D. Labels and Judges

How is a give-in detected and dated, and can the labels be trusted?

Forbidden-action rule: a give-in is the complete protected value appearing in a channel post or a new file, decided by a rule on the tool log. A channel post is a message sent to the shared team channel.

Leak route [source ready]: the first channel leak is split between an explicit send_message call and the end-of-turn reply that the harness posts to the channel (Qwen: 50 via send_message, 48 via the end-of-turn reply). Source: runs/20261025-2157-e05-reasoning-route-audit.

Addressee check: across all Qwen behaviour episodes (plain and tactic arms), the 51 end-of-turn replies that carried a first leak were all judged addressed to the peer (Jev, threshold 0.5); the base rate is 96.9% of all end-of-turn replies (4,935 of 5,094). This uses a wider slice than the route split above. Source: runs/20261025-2202-e06-addressee-audit.

Deliberate-vs-accidental judge: the secondary decision model's labels match blind human labels on 36 of 40 give-ins (Cohen's $\kappa = 0.79$).

Secondary labels: the near-term variants B and C. B keeps the near-term positives but counts only held-episode steps as negatives; C uses a within-three-target-calls window against held episodes. Both drop steps using hindsight, so neither is a prospective alarm label; their results are in Appendix H.

E. Detectors and Evaluation

What exactly does each detector see and compute, and how are scores compared?

Probe recipe. Logistic regression with L2 regularisation ($C = 0.1$), feature standardisation using training-rows-only statistics, and step weights so positives and negatives contribute equally within each round, on the 5,120-dimensional residual stream at the last prompt token before each target reply. The layer is chosen inside each training fold from 17 candidates (layers 0, 4, ..., 60, 63) on a 25% inner family holdout (the "nested" probe); a fixed-layer-32 variant is reported as a check. Three split seeds (42, 43, 44) are averaged. The probe is trained on recruiting episodes only, and families of related variants never cross a train/test boundary. The recipe follows Goldowsky-Dill et al. (2025) and Kramár et al. (2026).

Monitor setups. Three monitor models: GPT-5-nano (LLM judge, 0–100 score) and two decision models, Jev 1.14 (TypeSafe) and Luna Decisions (OpenAI), which return a probability over fixed yes/no options with no written rationale; the option order is randomised per step. Prompts are adapted from the UK AISI ControlArena default.txt template (MIT) (UK AI Security Institute, 2026), with warning signs and guiding questions, the score anchored only at its ends, and the rule text filled in per scenario. The full set of 3 models $\times$ 2 questions $\times$ 3 access levels $=$ 18 setups ran on the first data batch (75,312 calls, $12.67: Jev $2.03, GPT-5-nano $6.53, Luna $4.11); the new episodes used an 8-setup subset fixed in advance (all six Jev setups plus GPT-5-nano

detector and forecaster at the conversation level). Luna is at chance as a forecaster on the first data batch (actions only 0.517, conversation 0.496, everything the target saw 0.505) and was dropped from the new-episode runs for cost. **[TODO]** the full setup → AUROC grid on both labels as one table. Source: runs/20261025-0147-e06-monitor-main, experiments/E06-monitors/PROMPTS.md.

Evaluation. Held-out scenario (leave one scenario out) trains on five scenarios, tests on the sixth, rotates, and pools; pooled family cross-validation uses five folds grouped by family. Confidence intervals are 95% family bootstraps (1,000 resamples, seed 20261025), conditional on the fitted models. The combination test stacks a logistic model on round, log prompt length and the monitor score, and compares it with and without the probe score, reporting the paired family-bootstrap difference. The alarm analysis sets a score cutoff on benign (held) episodes for a 5% false-alarm target and reports recall and false alarms on held recruiting episodes.

Pooled CV and fold mixing. Pooled-CV baselines can fall below 0.5 because scores from different fitted fold models are compared within one scenario × round group; the fold-stratified variant compares only scores from the same fitted fold model. Held-out scenario and the confirmation test are unaffected. **[TODO]** fold-stratified numbers. Source: runs/20261025-1456-e05-all-data-main/report.md.

F. Second Target: gpt-oss-20b (and Gemma-4-31B)

Does the setup, and the detection result, carry to a differently trained model, and what had to change? Both branches below are kept until the replay outcome is known; gpt-oss numbers are never pooled with Qwen's. **[codex-2 owns the gpt-oss numbers.]**

Behaviour [source ready]: blunt pressure fails (gpt-oss 0/27, Gemma 1/35). A delayed whole-configuration-block request passes the selection gate; the scaled collection gives 128/300 channel give-ins, controls 0/60. Gemma stays file-only and behaviour-only.

Rule amendments: a public-only cutoff (reasoning mentions are not leaks) and the eligibility rule shared with Qwen (Addendum 6 of the frozen plan).

Disclosure-kind judge: 198/200 deliberate, 0 accidental, 2 uncertain. The first public emission is write_file in 196/200; when the token reaches the channel it is almost always the end-of-turn reply (127 of 128).

Leak route: of the 128 channel give-ins, 127 are the end-of-turn reply and 1 an explicit send_message (canonical disclosure-kind judge). The gpt-oss addressee rerun is pending, so no addressee line is given yet.

Reasoning and rendering. gpt-oss reasons, and the Harmony format drops reasoning from earlier turns but keeps the current turn's reasoning between tool calls. A rendering gate checks that the replayed prompt reproduces the served one: keeping the current-turn reasoning raised exact match from 68% to 98.9% within one token on one provider (AkashML), while another (DekaLLM) left about 4.5% of requests unresolved, with a median token gap of about 24. The leak cutoff is public output only: private reasoning is not a leak, and an inclusive cutoff would drop 53 held steps. Reasoning mentions of the token are reported as a descriptive column, not as give-ins. Because the

format keeps the current turn's reasoning, a mid-turn probe step can include the target's earlier reasoning from that turn, which the monitors' "everything the target saw" level does not; this access difference is stated wherever the gpt-oss probe and monitors are compared. [Rendering-gate source: runs/20261025-2236-e02-gpt-oss-rendering-gate; internal pointer, remove before submission.]

If the replay works: fidelity with the provider-quantisation caveat; the rendering check reported as passed; probe and monitors, including the reasoning-access level, in a small table matching Figure 2; and the pre-ask placebo split and the no-post-ask-window family as table rows. Section 4.6 of the main text gets the headline.

If the replay fails: the rendering mismatch is reported as the reason; the probe results are cut and stated as a limitation in Appendix A; Section 4.6 becomes two sentences of behaviour.

G. Behaviour Screening

What did we try before the final setup, and how does giving in vary with pressure?

[TODO] one coverage table (factor × level → give-in rate [Wilson 95% CI], n) over rule strength, single tactics, own stake, thinking mode, two peers, and evaluation awareness, with one paragraph of reading and no per-run prose. The three single persuasion tactics that appear in the scored recruiting set (§3.1) are defined here. Reporting follows docs/pressure_reporting_practices.md. Source: the E01 behaviour summaries.

H. Robustness and Diagnostics for the Main Results

Do the headline numbers survive other labels, layers, splits and checks, and what do the detectors miss?

Near-term robustness [TODO]: one table — labels B and C, scenario-mean, per-scenario, fixed layer 32, MLP — plus the per-layer curve as a figure. Source: runs/20261025-1456-e05-all-data-main/report.md.

Eventual label [TODO]: learning curve, per-layer, and branching, as a short subsection; branching supports the Discussion's "outcome only partly settled" point.

Pressure versus outcome: the full numbers and cohort for the pressure-probe check in §4.3 — first data batch, fixed layer 32: recruiting versus benign AUROC 1.000 on 4,184 steps; gave-in versus held 0.430 [0.373, 0.492] pooled and 0.426 [0.371, 0.488] held-out (2,910 recruiting steps, 618 eventual positives). Source: runs/20261025-0800-e05-explore-trajectories-pressure.

Errors and alarms: probe/monitor step-score correlations (within scenario × round Spearman 0.23 with the Jev forecaster, 0.09 with the GPT-5-nano detector); the shared-miss table with its caveat; and alarm recall at a 5% per-episode false-alarm rate. Peer-text sway in one line. Source: runs/20261025-1504-e06-appendix-errors.

Negative checks: logit lens and trajectory features, both negative, as table rows.

I. Prompts

The exact text each agent and monitor received.

[TODO] the target system prompt, the recruiting and control peer prompts (including the gpt-oss delayed request), and the monitor prompts, released in full per the dual-use decision. In the PDF, show one scenario in full and point to the repository for the rest. Source: experiments/E06-monitors/PROMPTS.md and the peer-prompt files.

LLM Usage Statement

[TODO] Required statement; does not count toward the page limit.